TalentLens AI

Candidate Ranking System

An AI-Powered Approach to Smarter Hiring

Generated: June 07, 2026

1. Problem Statement

Recruiters review hundreds of candidate profiles and often miss highly relevant candidates for specialized AI/ML roles. Traditional keyword-based filtering overlooks candidates with deep technical depth, non-obvious title matches, and diverse career paths that map to the job requirements.

The Challenge: How can we rank candidates the way a great recruiter would—by understanding who truly fits the role, not just matching keywords?

2. Solution Approach

TalentLens AI uses a **hybrid semantic + rule-based ranking system** that combines traditional talent signals with modern AI embeddings to evaluate candidate fit holistically.

Hybrid Ranking Framework

Component	Weight	Purpose
Rule-Based Scoring	40%	Title, experience, skills, career history, recruiter signals
Semantic Embeddings	60%	Deep understanding of job description and candidate profile ali

3. Scoring Components

A. Rule-Based Signals (40% weight):
- **Job Title Matching:** AI-aligned roles (+40), disqualifying roles (-60)
- **Experience Range:** 5-9 years target (+25), 4-11 years acceptable (+15)
- **Technical Skills:** Embeddings, retrieval, ranking, vector databases (+5 per match)
- **Career History:** Search engineer (+30), Recommendation systems (+35), ML roles (+25)
- **Recruiter Signals:** Response rate, open-to-work status, search activity

B. Semantic Matching (60% weight):
- Uses Sentence-Transformer embeddings (all-MiniLM-L6-v2 model)
- Embeds job description and candidate profiles into vector space
- Calculates cosine similarity for semantic relevance
- Captures nuanced role requirements beyond keyword matching

4. Ranking Algorithm

For each candidate:

1. Rule-Based Score (S_rule):
• Evaluate title, experience, skills, history, and recruiter signals
• Sum weighted scores across all matching criteria

2. Semantic Score (S_semantic):
• Embed job description: embedding(job_description)
• Embed candidate profile: embedding(profile + skills + history)
• Calculate similarity: cos_similarity(jd_embed, candidate_embed) × 100

3. Final Hybrid Score:
• Final Score = (S_rule × 0.4) + (S_semantic × 0.6)

4. Ranking:
• Sort candidates by Final Score in descending order
• Output top 100 ranked candidates to submission.csv

5. Data Pipeline

Input: Candidate data (candidates.jsonl) + Job description (job_description.docx)
Processing: Python ranking engine with embeddings and hybrid scoring
Output: Ranked candidates (submission.csv) with scores
Visualization: React dashboard for shortlist review and search

6. Output Format

The submission.csv file contains ranked candidates in CSV format:

candidate_id	rank	score
CAND_0018499	1	539.00
CAND_0092278	2	502.00
CAND_0033861	3	474.00
...	...	...

7. Key Advantages

Over Traditional Keyword Matching:
✓ Understands job requirements holistically, not just keywords
✓ Captures relevant experience even with non-standard titles
✓ Combines structured rules with semantic understanding
✓ Leverages recruiter signals (response rate, availability)
✓ Prioritizes production experience and demonstrated impact
✓ Transparent scoring (rule-based + semantic) for insights

8. How to Run

Backend (Generate Ranked Output):
cd ai-candidate-ranker/Backend/src
python rank_candidates.py

Copy Output to Frontend:
cp output/submission.csv ai-candidate-ranker/Frontend/public/submission.csv

Start Frontend Dashboard:
cd c:\Users\acer\OneDrive\Desktop\AI_Candidate_Ranker\ai-candidate-ranker
npm run preview

View Results:
Open the displayed localhost URL in your browser to explore ranked candidates.

9. Summary

TalentLens AI combines the best of both worlds: the structured, interpretable rules that capture domain knowledge about AI/ML hiring, and the power of modern semantic embeddings to understand nuanced fit. The hybrid approach ensures both precision (catching role-specific requirements) and recall (finding candidates with non-obvious but relevant backgrounds).